



Libraries
of
React

Libraries of React

れいな@底なし沼の魔女 著

2018-04-22 版 発行

はじめに

本書について

この本では React を使うにあたって作者が知り得た次の項目について、文書化したものです。

- 流行りに沿ったよりよいコードを書くための役立ったテクニック
- ゼロコンフィグ・モジュールバンドラーの知見
- 日本語情報が皆無だけど有益なモジュール

文書化して公開することで知見の棚卸しと React 開発者への共有を目的としています。React である程度開発を行なったことがある方向けの内容となっています。

注、React 初学者にはあまりオススメできません。

動作環境

本書では、次の環境にて操作を行っています。

- 動作確認 OS： Windows 10
- エディタ： Visual Studio Code（以下、VS Code）
- React： v16.3.1
- Node.js： v8.9.4
- npm： v5.6.0
- yarn： v1.5.1

一般的な開発者が普通に用意できる環境であることが前提になっています。（Windows 環境で動けば他でも問題ないと考えています）

プロジェクト構築手順

まずはじめに、実際に React のコードが動く足がかりとなるプロジェクトを作成します。

npx モジュールのインストール

環境によっては npx がインストールされていない場合があるので次のコマンドでインストールを実施します。

```
npm i npx -g
```

create-react-app でプロジェクト作成

npx モジュールから create-react-app を呼び出して React アプリのサンプルを作成します。

```
npx create-react-app react-sample
```

create-react-app を使用して作成されたプロジェクトをベースにして、サンプルを作成していきます。

目次

はじめに	2
本書について	2
動作環境	2
プロジェクト構築手順	3
npx モジュールのインストール	3
create-react-app でプロジェクト作成	3
第 1 章 Stateless Functional Component (SFC) で可読性向上	7
1.1 React のコンポーネントをおさらい	7
1.1.1 ES5 ベースの createClass を使用したコンポーネント (v15.5 より非推奨)	7
1.1.2 ES6 ベースで Component クラスを使用したコンポーネント (React v0.14 以降)	8
1.2 Stateless Functional Component (SFC)	8
1.3 SFC のメリット・デメリット	9
第 2 章 より良く React を使うテクニック	11
2.1 公式サイト「JSX In Depth」の JSX 記法	11
2.1.1 JSX のドット記法 (Using Dot Notation for JSX Type)	11
2.1.2 スプレッド属性 (Spread Attributes)	12
2.1.3 JavaScript の式を JSX の子要素で使う	13
2.1.4 false、Null、Undefined による表示判定処理 (Booleans, Null, and Undefined Are Ignored)	14
第 3 章 recompose で HOCs を扱う	16
3.1 recompose モジュールについて	16
3.2 recompose のインストール	17

3.2.1	state を付与する	17
3.2.2	イベントハンドラーを付与する	18
3.2.3	一部を別コンポーネントに切り出して Props を渡す	18
3.2.4	切り出したコンポーネントにライフサイクルイベントを付与	19
3.2.5	propTypes・defaultProps を付与する	21
3.3	recompose API 一覧	22
第 4 章	ゼロコンフィグ・モジュールバンドラー	25
4.1	React 開発時に必要な設定ファイル	25
4.2	ゼロコンフィグ・モジュールバンドラーとは	25
4.3	Facebook 公式モジュールバンドラー「react-scripts」	26
4.4	Parcel	26
4.4.1	概要	26
4.5	Parcel の基本的な使い方（最小構成のプロジェクト作成）	27
4.5.1	プロジェクト作成後、モジュールをインストール	27
4.5.2	起点となる HTML ファイル「index.html」を作成	27
4.5.3	HTML ファイルにインポートされている JavaScript ファイルを作成	28
4.5.4	package.json に開発サーバ起動用の npm スクリプト追加	28
4.5.5	開発サーバを起動するため、コマンド実行	29
4.6	コマンドライン設定	29
4.7	設定ファイル項目	29
4.7.1	BABEL	30
4.7.2	browserslist	30
4.7.3	uglify-config	31
4.7.4	.env	32
第 5 章	開発時のお勧めモジュール	33
5.1	styled-components	33
5.1.1	概要	33
5.1.2	create-react-app の CSS をすべて置き換える	33
	■コラム： styled-components のサーバサイドレンダリング（SSR）	44
あとがき		45
	お問い合わせ先	46
	免責事項	46

第 1 章

Stateless Functional Component (SFC) で可読性向上

1.1 React のコンポーネントをおさらい

コンポーネントを作成する際の書き方として次の 2 つの記法があります。

1.1.1 ES5 ベースの `createClass` を使用したコンポーネント (v15.5 より非推奨)

▼リスト 1.1 React のコンポーネントをおさらい (`createClass` コンポーネント)

```
// このソースコードは v16 以降では動作しません
var React = require("react");

var App = React.createClass({
  propTypes: {
    hoge: React.PropTypes.string
  },
  getDefaultProps: function() {
    return {
      hoge: ""
    };
  },
  getInitialState: function() {
    return {
      text: this.props.hoge || "hoge"
    };
  },
  render: function() {
    return <div>{this.state.text}</div>;
  }
});
```

1.1.2 ES6 ベースで Component クラスを使用したコンポーネント (React v0.14 以降)

▼リスト 1.2 React のコンポーネントをおさらい (クラス構文コンポーネント)

```
import React, { Component } from 'react';
import PropTypes from 'prop-types';

class App extends Component {
  static propTypes = {
    hoge: PropTypes.string
  };
  static defaultProps = {
    hoge: ''
  };
  state = {
    text: this.props.hoge || 'hoge hoge'
  };
  render() {
    return <div>{this.state.text}</div>;
  }
}

export default App;
```

1.2 Stateless Functional Component (SFC)

React 公式サイト (<https://reactjs.org>) を見ると、現在では Facebook の方針として、次のようなコーディングスタイルになっています。

- コンポーネントはシンプルに書く
- 必要がないなら state は持たず、Props のみでデータを管理する

という新しい概念のコンポーネント「Stateless Functional Component (SFC)」が主流になっています。

SFC を使うことによって、次のような開発スタイルが可能になっていきます。

- 「一部の処理を切り離したい」となった際に手軽にコンポーネントを作成できる
- state 情報を持たないので、Props 情報のみに注意を払える (人類には 2 つのデータを管理するのは早すぎるのです)

1.3 SFC のメリット・デメリット

そこで登場するのが Stateless Functional Component (SFC) です。

- state 情報を内部で保持しない
- Component クラスを継承しない

という特徴があります。名前からして前者のほうが重要に見えますが、後者のほうが重要であり次のメリット・デメリットが発生します。

- SFC のメリット
 - コンポーネントが View を返すのみの関数になるため可読性があがる
 - テスト時にテスト内容がシンプルになる
 - React v14 以降で高速化のチューニングがされている
- SFC のデメリット
 - Component クラスを継承しないため Context が使えない
 - React のライフサイクルメソッドが使えない
 - refs が使えない

実際にコードを書くと Component クラスが必要なケースのほうが圧倒的に少ないので、普段から SFC で書くように心がけるのをオススメします。SFC の記述方法については次のとおりです。

▼リスト 1.3 SFC の記述方法

```
// Component のインポートは不要
import React from 'react';
import PropTypes from 'prop-types';

// 関数でかける
const SfcComponent = props => {
  return <div>{props.text}</div>;
};

// 関数に対して propTypes を付与
SfcComponent.propTypes = {
  hoge: PropTypes.string.isRequired
};

// 関数に対して defaultProps を付与
SfcComponent.defaultProps = {
  text: 'hogehoge'
};
```

SFC を書いていて「ライフサイクルメソッドを使いたいけど class にするまでもない」という状況が生まれた際の対策として HOCs での実装を第 3 章「recompose で HOCs を扱う」で解説します。

第 2 章

より良く React を使うテクニック

2.1 公式サイト「JSX In Depth」の JSX 記法

React を使う上で一番コードを書くのは JSX。ということで、JSX について公式でまとめられているテクニックよりオススを解説していきます。

2.1.1 JSX のドット記法 (Using Dot Notation for JSX Type)

JSX のドット記法とは、ユーザ定義したコンポーネントにオブジェクト形式で 1 行 JSX を定義して使う記法です。

- オブジェクトリテラルにキーを指定することで、JSX の切り替えができる
- 1 行 JSX を複数書く場合に、グループ毎に纏めることができる

などのメリットがあります。実際の記法は次のとおりです。

▼リスト 2.1 ドット記法サンプル

```
import React from 'react';

const MyComponents = {
  Start: props => <div>Start: {props.message}</div>
};

function App() {
  return <MyComponents.Start message="hoge" />;
}

export default App;
```

この記法では MyComponents にキー情報を追加することでコンポーネントにキーを指定し、切り替えることができます。

▼リスト 2.2 ドット記法サンプル (複数キー)

```
import React from 'react';

const MyComponents = {
  Start: props => <div>Start: {props.message}</div>,
  End: props => <div>End: {props.message}</div>
};

function App() {
  return (
    <div>
      <MyComponents.Start message="hoge" />
      <MyComponents.End message="hoge" />
    </div>
  );
}

export default App;
```

2.1.2 スプレッド属性 (Spread Attributes)

ES2015 からのお馴染み、みんな大好きスプレッド属性です。

React では Props で設定することによりオブジェクトを展開した状態で Props として渡すことができます。

- ・ 親コンポーネントから渡ってきた Props をそのまま子コンポーネントに渡したい
- ・ 渡したい Props の数が多いので可読性をあげたい
- ・ コンポーネントの Props の記載量を減らす

などのメリットがあります。

実際の記法は次のとおりです。

▼リスト 2.3 スプレッド属性サンプル

```
import React from 'react';
const ChildApp = props => {
  return (
    <div>{`firstName: ${props.firstName} lastName: ${props.lastName}`}</div>
  );
};

const App = () => {
  const props = { firstName: 'hoge', lastName: 'fuga' };
  return <ChildApp {...props} />;
};

export default App;
```

2.1.3 JavaScript の式を JSX の子要素で使う

JSX ですがタグ内で「{ }」と書くことで、その内部で JavaScript の式を使用できます。

▼リスト 2.4 子要素に変数を展開

```
const hoge = 'hoge hoge';
<MyComponent>{foo}</MyComponent>
```

子要素に JavaScript の式を使用できるので次のような無名関数を使用することも可能です。

▼リスト 2.5 子要素で無名関数を使う

```
import React, { Component } from 'react';

/**
 * リスト要素を出力する
 * @param {*} props message と key を含む
 */
const Item = props => {
  // 引数の Props のメッセージ表示
  return <li>{props.message}</li>;
};

/**
 * リスト表示
 */
const TodoList = () => {
  const todos = ['finish doc', 'submit pr', 'nag dan to review'];
  return (
    // map で無名関数によって Item コンポーネントが出力される
    <ul>{todos.map(message => <Item key={message} message={message} />)}</ul>
  );
};

class App extends Component {
  render() {
    return <div>{TodoList()}</div>;
  }
}

export default App;
```

上記のサンプルでは子要素コンポーネントに対して Props で関数を渡していますが、子要素として関数を渡すことができます。

実際の記法は次のとおりです。

▼リスト 2.6 無名関数を渡して props.children で実行する

```
import React, { Component } from 'react';

/**
 * Props の numTimes に指定回数で子要素の関数を実行
 * @param {*} props Props には numTimes と children に設定された無名関数が来る
 */
const Repeat = props => {
  let items = [];
  for (let i = 0; i < props.numTimes; i++) {
    // props.children(引数) とすることで子要素して渡された式を実行する
    items.push(props.children(i));
  }
  // 最後に JSX を格納した配列を {} 内で展開することで作成した JSX を展開する
  return <div>{items}</div>;
};

/**
 * Repeat 関数を呼び出す関数
 */
const FunctionsAsChildren = () => {
  return (
    <Repeat numTimes={10}>
      {/* 子要素として引数をもとに JSX を返す無名関数を渡している */}
      {index => <div key={index}>This is item {index} in the list</div>}
    </Repeat>
  );
};

class App extends Component {
  render() {
    return <div>{FunctionsAsChildren()}</div>;
  }
}

export default App;
```

この記法は、後の章で出てくる HOCs と v16.3 で実装された新しい Context API を扱う際に重要となります。ガンガン素振りをしておきましょう。

2.1.4 false、Null、Undefined による表示判定処理 (Booleans, Null, and Undefined Are Ignored)

JSX の子要素内に false として扱われる値があった場合、以降の文字列が表示されない仕組みになっています。

この仕組みを使うことで、ワンライナーで JSX 表示の有無ができるようになります。実際の記法は次のとおりです。

boolean による表示判定

▼リスト 2.7 JSX 内での表示判定処理サンプル

```
import React from 'react';
const ChildApp = props => {
  return <div>{props.show && <div>hoge hoge</div>}</div>;
};

const App = () => {
  return <ChildApp show />;
};

export default App;
```

このサンプルでは、「props.show」が true の場合「<div>hoge hoge</div>」が表示される仕組みになっています。

文字列の長さによる表示判定

JavaScript では 0 も false として扱われるので文字数チェックして、0 でなければ表示するということができます。

実際の記法は次のとおりです。

▼リスト 2.8 JSX 内での表示判定処理サンプル（メッセージ文字数 0 なら非表示にする）

```
import React from 'react';
const ChildApp = props => {
  return <div>{props.message.length && <div>{props.message}</div>}</div>;
};

const App = () => {
  return <ChildApp message="hoge hoge" />;
};

export default App;
```

この記法も JSX をシンプルに書くにはとても有効なものですので、ガンガン素振りをして使っていきましょう。

第 3 章

recompose で HOCs を扱う

Higher-Order Components (HOCs^{*1}) について、公式サイト (<https://reactjs.org/docs/higher-order-components.html>) の説明は次のとおりです。

Concretely, a higher-order component is a function that takes a component and returns a new component. (具体的には、HOCs とは Props として受け取ったコンポーネントを新たなコンポーネントとして返すことです。)

これは次のようにとらえることができます。

- 作成したコンポーネントに処理を追加するためのラッパー
- 複数の処理をまとめるための Mixin (多重継承) ← この考えが後々重要になってきます

実際に書いていくのはそれなりに面倒です。その手間を省くモジュールとして「recompose」を紹介します。

3.1 recompose モジュールについて

公式サイト (<https://github.com/acdlite/recompose>) 引用。

Recompose is a React utility belt for function components and higher-order components. Think of it like lodash for React. (Recompose は React で HOCs をするための工作ベルトです。React 版 lodash みたいなものと考えてください。)

^{*1} Higher-Order Components の略称について本書では「HOCs」で統一しています。

3.2 recompose のインストール

次のコマンドを実行してモジュールを追加します。

```
yarn add recompose
```

recompose の代表的なコンポーネント（ここでは API と呼びます）を使ったサンプルコードを上げてきます。

サンプルについては、create-react-app で作成した index.js を使用します。

実際の記法は次のとおりです。

3.2.1 state を付与する

create-react-app で作成した index.js は state 情報を持ちません。まずは state を付与してみましょう。

実際の記法は次のとおりです。

▼リスト 3.1 HOCs サンプル (withState の実装)

```
import React, { Component } from 'react';
import { compose, withState } from 'recompose';
import logo from './logo.svg';
import './App.css';

// HOCs の関数を定義します compose は複数の関数を 1 つにまとめる際に使用します
const enhanceApp = compose(withState('counter', 'setCounter', 0));

class App extends Component {
  render() {
    return (
      <div className="App">
        <header className="App-header">
          <img src={logo} className="App-logo" alt="logo" />
          <h1 className="App-title">Welcome to React</h1>
        </header>
        <p className="App-intro">
          To get started, edit <code>src/App.js</code> and save to reload.
        </p>
        <input type="button" value={this.props.counter} />
      </div>
    );
  }
}

export default enhanceApp(App);
```

3.2.2 イベントハンドラーを付与する

state 情報があっていい感じになりましたね。次に state を変更するためのクリックイベントに対してイベントハンドラーをつけてみましょう。

実際の記法は次のとおりです。

▼リスト 3.2 HOCs サンプル (withHandlers の実装)

```
import React, { Component } from 'react';
import { compose, withState, withHandlers } from 'recompose';
import logo from './logo.svg';
import './App.css';

const enhanceApp = compose(
  withState('counter', 'setCounter', 0),
  withHandlers({
    increment: ({ setCounter }) => () => setCounter(n => n + 1)
  })
);

class App extends Component {
  render() {
    return (
      <div className="App">
        <header className="App-header">
          <img src={logo} className="App-logo" alt="logo" />
          <h1 className="App-title">Welcome to React</h1>
        </header>
        <p className="App-intro">
          To get started, edit <code>src/App.js</code> and save to reload.
        </p>
        <input
          type="button"
          value={this.props.counter}
          onClick={this.props.increment}
        />
      </div>
    );
  }
}

export default enhanceApp(App);
```

3.2.3 一部を別コンポーネントに切り出して Props を渡す

これで「クリックするとカウントアップする」ができました。ここでボタン部分を SFC で切り出してみましょう。

実際の記法は次のとおりです。

▼リスト 3.3 HOCs サンプル (ステートレスコンポーネントの切り出し)

```
import React, { Component } from 'react';
import { compose, withState, withHandlers } from 'recompose';
import logo from './logo.svg';
import './App.css';

const enhanceApp = compose(
  withState('counter', 'setCounter', 0),
  withHandlers({
    increment: ({ setCounter }) => () => setCounter(n => n + 1)
  })
);

const Btn = ({ increment, counter }) => {
  return (
    <div>
      <input type="button" value={counter} onClick={increment} />
    </div>
  );
};

class App extends Component {
  render() {
    return (
      <div className="App">
        <header className="App-header">
          <img src={logo} className="App-logo" alt="logo" />
          <h1 className="App-title">Welcome to React</h1>
        </header>
        <p className="App-intro">
          To get started, edit <code>src/App.js</code> and save to reload.
        </p>
        <Btn {...this.props} />
      </div>
    );
  }
}

export default enhanceApp(App);
```

3.2.4 切り出したコンポーネントにライフサイクルイベントを付与

Btn コンポーネントですが、このままだと親コンポーネントから Props を受け取ると毎回 Render が走ってしまいます。

パフォーマンスチューニングとして shouldUpdate をつけて、Props の更新がなければ、Render しないようにしてみましょう。

実際の記法は次のとおりです。

▼リスト 3.4 HOCs サンプル (shouldUpdate でライフサイクル実装)

```
import React, { Component } from 'react';
import {
  compose,
  shouldUpdate,
  shallowEqual,
  withState,
  withHandlers
} from 'recompose';
import logo from './logo.svg';
import './App.css';

const enhanceApp = compose(
  withState('counter', 'setCounter', 0),
  withHandlers({
    increment: ({ setCounter }) => () => setCounter(n => n + 1)
  })
);

const enhanceBtn = compose(
  shouldUpdate((props, nextProps) => {
    shallowEqual(props, nextProps);
  })
);

const Btn = ({ increment, counter }) => {
  return (
    <div>
      <input type="button" value={counter} onClick={increment} />
    </div>
  );
};

class App extends Component {
  render() {
    const EnhanceBtn = enhanceBtn(Btn);
    return (
      <div className="App">
        <header className="App-header">
          <img src={logo} className="App-logo" alt="logo" />
          <h1 className="App-title">Welcome to React</h1>
        </header>
        <p className="App-intro">
          To get started, edit <code>src/App.js</code> and save to reload.
        </p>
        <EnhanceBtn {...this.props} />
      </div>
    );
  }
}

export default enhanceApp(App);
```

3.2.5 propTypes・defaultProps を付与する

かなりアプリっぽくなってきましたが、develop モードで起動すると常時 PropTypes が無いと怒られるのも気分が良くないですね。

defaultProps、setPropTypes もどーんとつけてみましょう。

実際の記法は次のとおりです。

▼リスト 3.5 HOCs サンプル (defaultProps、setPropTypes の実装)

```
import React, { Component } from 'react';
import PropTypes from 'prop-types';
import {
  compose,
  defaultProps,
  setPropTypes,
  shouldUpdate,
  shallowEqual,
  withState,
  withHandlers
} from 'recompose';
import logo from './logo.svg';
import './App.css';

const enhanceApp = compose(
  withState('counter', 'setCounter', 0),
  withHandlers({
    increment: ({ setCounter }) => () => setCounter(n => n + 1)
  })
);

const enhanceBtn = compose(
  setPropTypes({
    counter: PropTypes.number.isRequired,
    increment: PropTypes.func.isRequired
  }),
  defaultProps({
    counter: 0,
    increment: () => console.log('increment')
  }),
  shouldUpdate((props, nextProps) => {
    shallowEqual(props, nextProps);
  })
);

const Btn = ({ increment, counter }) => {
  return (
    <div>
      <input type="button" value={counter} onClick={increment} />
    </div>
  );
};

class App extends Component {
```

```
render() {
  const EnhanceBtn = enhanceBtn(Btn);
  return (
    <div className="App">
      <header className="App-header">
        <img src={logo} className="App-logo" alt="logo" />
        <h1 className="App-title">Welcome to React</h1>
      </header>
      <p className="App-intro">
        To get started, edit <code>src/App.js</code> and save to reload.
      </p>
      <EnhanceBtn {...this.props} />
    </div>
  );
}
}

export default enhanceApp(App);
```

皆さんここまでお疲れ様でした。今度は、実際に実装してみて recompose の良さを体験してみてください。

今回は代表的な API のみでしたが、実際には数多くの API があります。公式サイトの API リファレンス (<https://github.com/acdlite/recompose/blob/master/docs/API.md>) を参照して HOCs ライフを充実させましょう。

3.3 recompose API 一覧

recompose API について、概要を纏めたリストとなります。

具体的な使用方法是公式サイトを参照してください。

▼表 3.1 HOCs 実装時に主に使用する API

API 名	概要
<code>mapProps()</code>	引数の Props を加工して Props を HOCs に渡す
<code>withProps()</code>	引数の Props を加工して Props に追加して HOCs に渡す
<code>withPropsOnChange()</code>	変更時のみ <code>withProps</code> を実行して Props を HOCs に渡す
<code>withHandlers()</code>	イベントハンドラーのメソッドを Props として HOCs に渡す
<code>defaultProps()</code>	<code>defaultProps</code> を設定して HOCs に渡す
<code>renameProp()</code>	Props のキー名を 1 つだけ変更して HOCs に渡す
<code>renameProps()</code>	Props のキー名を複数項目変更して HOCs に渡す
<code>flattenProp()</code>	Props 内のオブジェクトリテラルを展開して HOCs に渡す
<code>withState()</code>	state を付与する。変更する関数とデフォルト値を設定可能
<code>withStateHandlers()</code>	state を付与する。複数の関数を設定可能
<code>withReducer()</code>	[Redux]Reducer を付与する。
<code>branch()</code>	第 1 引数 true:第 2 引数、第 1 引数 false:第 3 引数を返す
<code>renderComponent()</code>	引数で渡されたコンポーネントを render する
<code>renderNothing()</code>	引数で渡されたコンポーネントを render しないで Null を返す
<code>shouldUpdate()</code>	<code>shouldComponentUpdate</code> 処理を実装する
<code>pure()</code>	[React]PureComponent と同様の処理を実装する
<code>onlyUpdateForKeys()</code>	引数の配列にある Props のキー更新時のみ再 render する
<code>onlyUpdateForPropTypes()</code>	<code>propTypes</code> にある Props のキー更新時のみ再 render する
<code>withContext()</code>	[Redux]store を付与して Provider コンポーネントにする
<code>getContext()</code>	[Redux]Provider コンポーネントの store 情報を取得する
<code>lifecycle()</code>	各ライフサイクルメソッドを定義する
<code>toClass()</code>	クラス構文化して refs を使えるようにする

▼表 3.2 Props 周りを設定する際に使用する API

API 名	概要
<code>setStatic()</code>	クラス構文の static プロパティを定義する
<code>setPropTypes()</code>	クラス構文の static propTypes を定義する
<code>setDisplayName()</code>	コンポーネントの displayName を定義する

▼表 3.3 recompose 内でユーティリティとして使用する API

API 名	概要
<code>compose()</code>	【重要】複数の API を 1 つにする。大抵これを使う
<code>getDisplayName()</code>	コンポーネントの <code>displayName</code> を取得する
<code>wrapDisplayName()</code>	引数の React コンポーネントを <code>displayName</code> でラップして返す
<code>shallowEqual()</code>	2 つのオブジェクトリテラルの浅い比較をする
<code>isClassComponent()</code>	引数がクラス構文のコンポーネントかを判定し、Boolean を返す
<code>createSink()</code>	<code>render</code> を行っていないコンポーネントを返す
<code>componentFromProp()</code>	引数の <code>prop</code> 名がもつ値からコンポーネントを作成して返す
<code>nest()</code>	渡されたコンポーネントを第 1 引数からネストにして返す
<code>hoistStatics()</code>	非 React コンポーネントの Static プロパティを HOCs 対象にコピー

▼表 3.4 RxJS の Observable を扱う際にユーティリティとして使用する API

API 名	概要
<code>componentFromStream()</code>	Stream を加工して Props の Stream を HOCs に渡す
<code>componentFromStreamWithConfig()</code>	Config を設定して <code>componentFromStream</code> 実施
<code>mapPropsStream()</code>	Props の Stream を加工して HOCs に渡す
<code>mapPropsStreamWithConfig()</code>	Config を設定して <code>mapPropsStreamWithConfig</code> 実施
<code>createEventHandler()</code>	handler プロパティと stream プロパティを返す
<code>createEventHandlerWithConfig()</code>	Config を設定して <code>createEventHandler</code> 実施
<code>setObservableConfig()</code>	ES Observable proposal を RxJS にするための設定

第 4 章

ゼロコンフィグ・モジュールバンドラー

4.1 React 開発時に必要な設定ファイル

React にて開発する上で使う設定ファイルは次のとおりです。

- 【必須】 Javascript をトランスパイルする BABEL 設定 (`.babelrc`)
- 【必須】 必要なモジュールを纏めるための Webpack の設定 (`webpack.config.js`)
- 静的検証ツール (ESLint) のルール設定 (`.eslintrc.json`)
- コードフォーマッターのルール設定 (`.prettierrc`)
- 型定義する場合は、FlowType 設定 (`.flowconfig`)

それぞれの設定はある程度テンプレート化していても面倒な作業になっています。

必須ファイルの `.babelrc` と `webpack.config.js` については、その柔軟性と選択できるモジュールの量から初見殺しになりがちです。

4.2 ゼロコンフィグ・モジュールバンドラーとは

選択肢の多さで、コーディング以外のところで体力を使う辛い状況から「設定ファイルを 0 にして実際の開発に注力する」モジュールが数多く生まれました。

それらは「ゼロコンフィグ・モジュールバンドラー」と呼ばれ、次の項目を満たしているモジュールを指しています。

- `.babelrc` なしで BABEL によってトランスパイルが行われること
- `webpack.config.js` が無くてもパッケージとしてまとめられること
- 開発用サーバやリリース用のパッケージ作成のコマンドが用意されていること

- 設定ファイルについて最小限で動作可能であること

これらのゼロコンフィグ・モジュールバンドラーでまず思いつくのが「create-react-app」でしょう。

4.3 Facebook 公式モジュールバンドラー「react-scripts」

react-scripts は create-react-app でプロジェクトを作成した際にインストールされるモジュールです。webpack.config.jsや.babelrcは不要で、react-scripts が全て設定してくれます。

React を始めた頃や素振りをする際には大変有効なツールなのですが、本格的にプロジェクトを進めると次の問題に多かれ少なかれ直面すると思われます。

- BABEL プラグインの追加ができない
- Webpack 設定ができないため、必要な loader を追加できない
- ESLint のルールを変更できない
- eject することで設定可能になるが元に戻せない
- eject した設定ファイルが複雑で変更するのが大変

React で開発している人にとって共通の悩みとなっていました。最近になり柔軟なモジュールバンドラーが続々と開発されるようになりました。

この章では、利便性と設定の自由度が高いツールとして「Parcel」を紹介します。

4.4 Parcel

4.4.1 概要

Adobe 社の Devon Govett 氏によって開発されたモジュールバンドラーです。主な特徴として公式サイト <https://parceljs.org/> からの引用。

- Blazing fast bundle times - multicore compilation, and a filesystem cache for fast rebuilds even after a restart.
- Out of the box support for JS, CSS, HTML, file assets, and more - no plugins to install.
- Automatically transforms modules using Babel, PostCSS, and PostHTML when needed - even node_modules.
- Zero configuration code splitting using dynamic import() statements.
- Built in support for hot module replacement

- Friendly error logging experience - syntax highlighted code frames help pinpoint the problem.

ざっくりまとめると「早い・楽ちん・拡張しやすい」を三拍子揃ったモジュールバンドラーとなっております。作者が使ってみて感じたメリット・デメリットは次のとおりです。

- メリット
 - parcel-bundler のみで動作する
 - 既存の.babelrc や.env などの設定ファイルを流用できる
 - エントリポイントとインポートされる JavaScript ファイルがあればいい
- デメリット
 - ビルド時に vendor ファイルを含めて 1 ファイルに結合されてしまう
 - static フォルダが未実装なので静的ファイルはすべて import が必須
 - 公式のドキュメントが少なく、Issue とソースをチェックする必要がある

それなりにデメリットもありますが、とにかくサクサク動くという魅力がすごいので、ちょっとでも気になったら次の項に進んで素振りをしてみましょう。

4.5 Parcel の基本的な使い方（最小構成のプロジェクト作成）

4.5.1 プロジェクト作成後、モジュールをインストール

```
// プロジェクト初期化
yarn init

// npm モジュール追加
yarn add parcel-bundler react react-dom
```

4.5.2 起点となる HTML ファイル「index.html」を作成

▼リスト 4.1 index.html

```
<!DOCTYPE html>
<html lang="ja">

<head>
  <meta charset="utf-8">
  <title>Parcel Simple</title>
</head>

<body>
  <div id="root"></div>
  <script src="./index.js"></script>
</body>

</html>
```

4.5.3 HTML ファイルにインポートされている JavaScript ファイルを作成

▼リスト 4.2 index.js

```
import React from 'react';
import ReactDOM from 'react-dom';

const App = () => <h1>Parcel Simple</h1>;

ReactDOM.render(<App />, document.getElementById('root'));
```

4.5.4 package.json に開発サーバ起動用の npm スクリプト追加

▼リスト 4.3 package.json

```
{
  "name": "2018-04-22-parcel-sample",
  "version": "1.0.0",
  "main": "index.js",
  "repository": "git@github.com:MegaBlackLabel/2018-04-22-parcel-sample.git",
  "author": "底なし沼の魔女 <megablacklabel@gmail.com>",
  "license": "MIT",
  "scripts": {
    "start": "parcel ./index.html"
  },
  "dependencies": {
    "parcel-bundler": "^1.7.0",
    "react": "^16.3.1",
    "react-dom": "^16.3.1"
  }
}
```

4.5.5 開発サーバを起動するため、コマンド実行

```
yarn start
```

開発用のローカルアドレス `http://localhost:1234/` を開く。



▲図 4.1 Parcel の開発モード起動

v1.62 では `babel-preset-env`、`babel-preset-react` の取込が行われており、別途 `.babelrc` での設定は不要となっています。

4.6 コマンドライン設定

`parcel` は起動時にコマンドライン設定が可能です。設定項目は次のとおり。

4.7 設定ファイル項目

`Parcel` は基本的に設定ファイル無しですが、ルートフォルダに配置することで外部ファイルを読み込むことができます。設定できる項目は次のとおりです。

▼表 4.1 コマンドライン設定項目

コマンド名	設定内容	初期値（設定可能値）
-p, --port <port>	ポート番号	1234
--hmr-port <port>	HMR の Websockets ポート番号	ランダム
--hmr-hostname <hostname>	HMR の Websockets ホスト名	ローカルホスト名
--https	HTTPS 使用	未設定時は HTTP 使用
--open	ビルド時ブラウザを開く	ブラウザを開かない
-d, --out-dir <path>	ビルドファイル出力先	/dist
-o, --out-file <filename>	ビルド時のエントリポイント名	index.html
--public-url <url>	ビルド時の public フォルダ	--out-dir と同一
--no-hmr	HMR を使用しない	HMR 使用
--no-cache	cache を使用しない	cache を作成
--no-source-maps	Sourcemaps を作成しない	SourceMaps 作成
-t, --target [target]	ビルド時のターゲット	browser (node, browser, electron)
--no-minify	未設定時は難読化をする (プロダクション作成時のみ有効)	難読化を行う

4.7.1 BABEL

.babelrc (.babelrc.json、.babelrc.yaml) もしくは package.json で設定可能です。
babel-preset-env はインストール不要で使用可能です。

4.7.2 browserslist

.browserslistrc もしくは package.json で設定可能です。browserslist はインストール不要で次の手順で使用できます。

package.json に追記

ブラウザ条件は次のとおりです。

- 各ブラウザの最新から 2 バージョンまでを対象
- IE のバージョンが 11 以上

▼リスト 4.4 package.json

```
{
  "name": "parcel-simple",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "parcel ./index.html --open"
  },
  "license": "MIT",
  "dependencies": {
    "parcel-bundler": "^1.6.2",
    "react": "^16.2.0",
    "react-dom": "^16.2.0"
  },
  "browserslist": [
    "> 1%",
    "last 2 versions",
    "IE 11"
  ]
}
```

.browserslistrc で設定

▼リスト 4.5 .browserslistrc

```
> 1%
Last 2 versions
IE 11 # IEは11以上
```

内部でコメントが使えるので.browserslistrcのほうが便利です。

4.7.3 uglify-config

.uglifyrc で設定可能。uglify-es はインストール不要で使用可能となっています。

次の設定で minify 時に console.log を削除します。

▼リスト 4.6 .uglifyrc

```
{
  "compress": {
    "drop_console": true
  }
}
```

これでプロダクションとして dist 出力時に console.log が削除されて出力されます。

4.7.4 .env

.env で設定可能。

.env に定数を設定する

▼リスト 4.7 .env の設定サンプル

```
HOG=hogehoge  
FUGA=fugafuga
```

Parcel を使用しているプロジェクトで.env の定数を使用するサンプル

▼リスト 4.8 .env を使った判定処理サンプル

```
if (process.env.HOG === 'hogehoge') {  
  console.log(process.env.HOG);  
}
```

Parcel はコミュニティが活発で、これから安定した開発が継続されそうなので一度は素振りをしてみてください。

第 5 章

開発時のお勧めモジュール

5.1 styled-components

5.1.1 概要

みんな大好き「CSS in JS」の中でも一番人気（と思われる）の「styled-components」ですが次のような特徴を持っています。

- CSS in JS を実装するモジュール。既存の HTML タグを拡張して JavaScript 内で CSS を記述できる
- JavaScript で実装するため、JavaScript の記法や Props を利用できる
- 1 コンポーネントとして扱える
- 実態としては、head タグ内に style data-styled-components タグを追加して内部にスタイル情報を保持している
- React Native や Server Side Rendering にも対応している

これだけだと分かりにくいですね。実際に実装して行って、その凄さを見ていきましょう。

5.1.2 create-react-app の CSS をすべて置き換える

今回は「create-react-app」で作ったアプリの初期状態から、外見を変えずに CSS in JS で置き換えてみます。

まずはじめに次のコマンドを実行してプロジェクトを作成します。

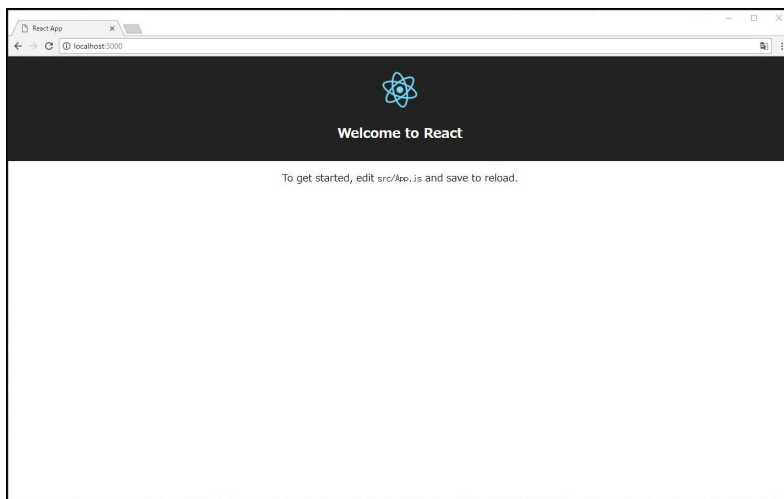
```
npx create-react-app 2018-04-22-parcel-styled-components-sample
```

VS Code でプロジェクトを開き、「ターミナル」から styled-components モジュールを追加します。

```
// npm モジュール追加  
yarn add styled-components
```

「ターミナル」から開発サーバを起動します。

```
yarn start
```



▲図 5.1 create-react-app 初期画面

global CSS の実装

はじめにグローバル CSS の置き換えを行います。

コードを次のように修正します。

▼リスト 5.1 ./src/index.js

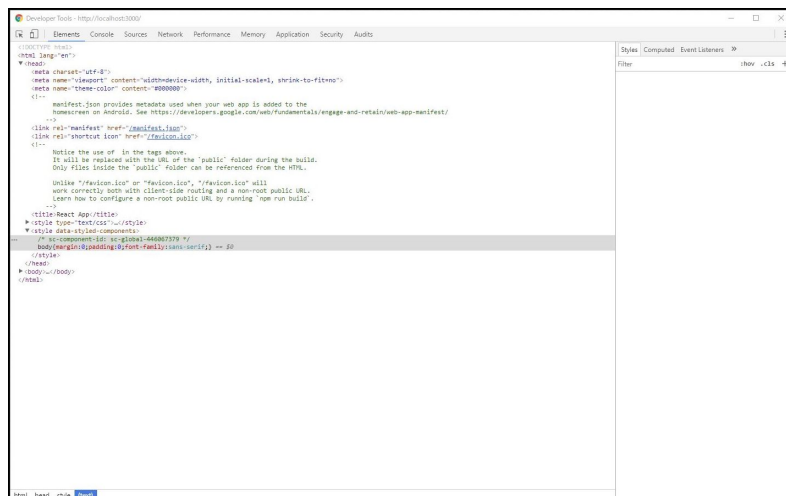
```
import React from 'react';  
import ReactDOM from 'react-dom';  
// import './index.css';  
import App from './App';
```

```
import { injectGlobal } from 'styled-components';
import registerServiceWorker from './registerServiceWorker';

ReactDOM.render(<App />, document.getElementById('root'));
registerServiceWorker();

injectGlobal`
  body {
    margin: 0;
    padding: 0;
    font-family: sans-serif;
  }
`;
```

置き換え完了後、ブラウザが再描画されます。そこで Chrome Dev Tools を使ってコードを見えます。



▲図 5.2 Chrome Dev Tools

このように injectGlobal で追加した CSS がヘッドに追加されます。

JSX のコンポーネントへの実装

次に app.js にある CSS を置き換えてみます。

コードを次のように修正します。

▼リスト 5.2 ./src/app.js

```
import React, { Component } from 'react';
import styled, { keyframes } from 'styled-components';
import logo from './logo.svg';
// import './App.css';

const AppWrapper = styled.div`
  text-align: center;
`;

const AppHeader = styled.header`
  background-color: #222;
  height: 150px;
  padding: 20px;
  color: white;
`;

const AppLogoSpin = keyframes`
  from { transform: rotate(0deg); }
  to { transform: rotate(360deg); }
`;

const AppLogo = styled.img`
  animation: ${AppLogoSpin} infinite 20s linear;
  height: 80px;
`;

const AppTitle = styled.h1`
  font-size: 1.5em;
`;

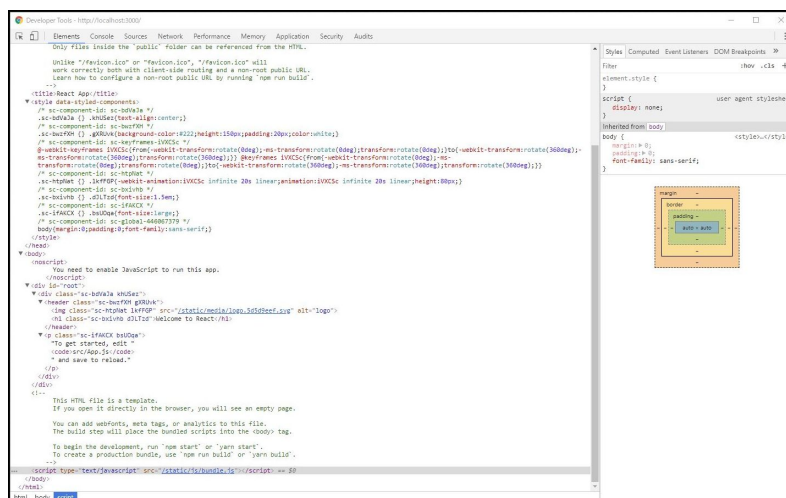
const AppIntro = styled.p`
  font-size: large;
`;

class App extends Component {
  render() {
    return (
      <AppWrapper>
        <AppHeader>
          <AppLogo src={logo} alt="logo" />
          <AppTitle>Welcome to React</AppTitle>
        </AppHeader>
        <AppIntro>
          To get started, edit <code>src/App.js</code> and save to reload.
        </AppIntro>
      </AppWrapper>
    );
  }
}

export default App;
```

修正後、ブラウザが再描画されます。そこで Chrome Dev Tools を使ってコードを見えます。

同じようにヘッダに CSS が追加されますが、今回はクラス名付きの CSS が追加されます。このクラス名と各タグの class とが紐付いて CSS が適用されます。



▲図 5.3 style タグと class 名が連動

Props の実装 (Props によってフォントの太さを変更する)

styled-components はコンポーネントが受け取った Props を使って内部で処理を実装することもできます。

今度はコードを次のように修正します。

▼リスト 5.3 ./src/app.js

```
import React, { Component } from 'react';
import styled from 'styled-components';
import logo from './logo.svg';
import './App.css';

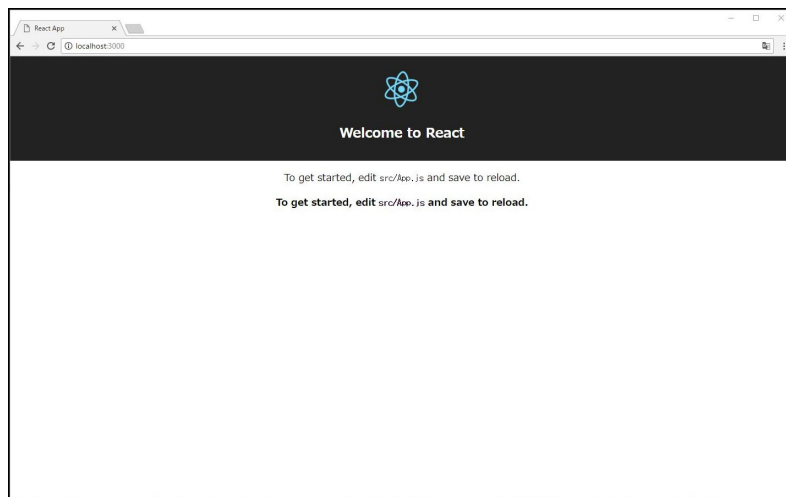
const AppIntro = styled.p`
  font-size: large;
  // 関数内では文字列扱いなのでシングルコーテーションで括る
  font-weight: ${props => (props.primary ? 'bold' : 'normal')};
`;

class App extends Component {
  render() {
    return (
      <div className="App">
        <header className="App-header">
          <img src={logo} className="App-logo" alt="logo" />
          <h1 className="App-title">Welcome to React</h1>
        </header>
        <AppIntro>

```

```
    To get started, edit <code>src/App.js</code> and save to reload.  
  </AppIntro>  
  <AppIntro primary>  
    To get started, edit <code>src/App.js</code> and save to reload.  
  </AppIntro>  
</div>  
);  
}  
}  
  
export default App;
```

修正後にブラウザを表示すると太さの異なる文字が 2 つ表示されます。これは Props によって font-weight を設定できるからです。



▲ 図 5.4 font-weight が異なる 2 つのコンポーネントを表示

hover の実装

styled-components でちょっと複雑な CSS の実装をしてみます。今回は hover を取り扱っています。

今度はコードを次のように修正します。

▼ リスト 5.4 ./src/app.js

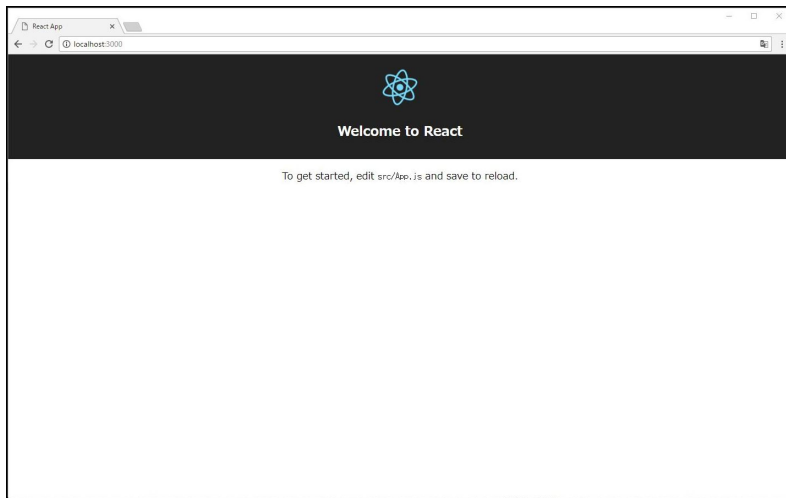
```
import React, { Component } from 'react';
import styled from 'styled-components';
import logo from './logo.svg';
import './App.css';

const AppHeader = styled.header`
  background-color: #222;
  height: 150px;
  padding: 20px;
  color: white;
  &:hover {
    background-color: white;
    color: #222;
  }
`;

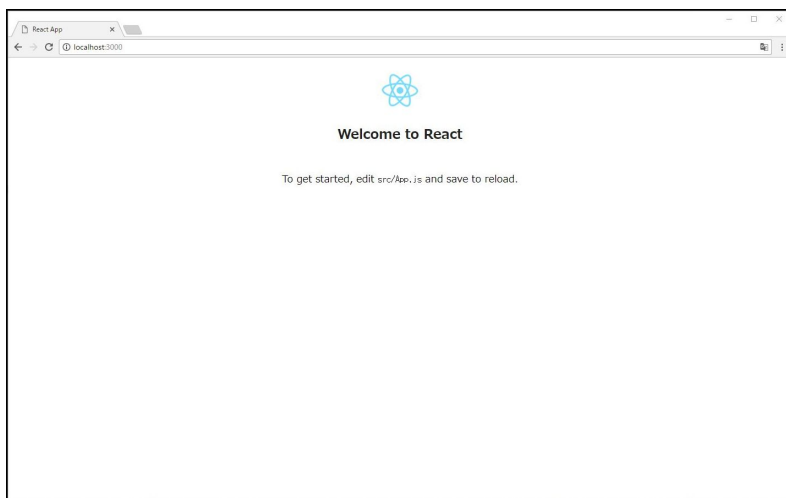
class App extends Component {
  render() {
    return (
      <div className="App">
        <AppHeader>
          <img src={logo} className="App-logo" alt="logo" />
          <h1 className="App-title">Welcome to React</h1>
        </AppHeader>
        <p className="App-intro">
          To get started, edit <code>src/App.js</code> and save to reload.
        </p>
      </div>
    );
  }
}

export default App;
```

修正後にブラウザを表示すると hover でヘッダ部の背景色が変化をします。



▲ 図 5.5 ヘッダ部の背景色が hover で変化 (hover 前)



▲ 図 5.6 ヘッダ部の背景色が hover で変化 (hover 後)

実装済みのタグへの追加実装

styled-components でコンポーネントを作っていくと「ベースコンポーネントにちょっと追加したい」という要望にも対応できます。

app.js を次のように修正します。

▼リスト 5.5 ./src/app.js

```
import React, { Component } from 'react';
import styled from 'styled-components';
import logo from './logo.svg';
import './App.css';

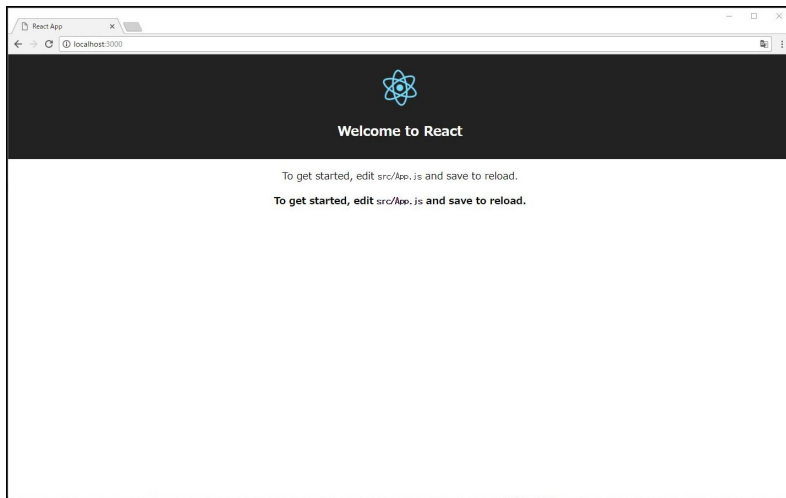
const AppIntro = styled.p`
  font-size: large;
`;

const StyledIntro = AppIntro.extend`
  font-weight: bold;
`;

class App extends Component {
  render() {
    return (
      <div className="App">
        <header className="App-header">
          <img src={logo} className="App-logo" alt="logo" />
          <h1 className="App-title">Welcome to React</h1>
        </header>
        <AppIntro>
          To get started, edit <code>src/App.js</code> and save to reload.
        </AppIntro>
        <StyledIntro>
          To get started, edit <code>src/App.js</code> and save to reload.
        </StyledIntro>
      </div>
    );
  }
}

export default App;
```

修正後にブラウザを表示すると extend によって CSS を追加しているため太さの異なるの文字が 2 つ表示されます。



▲図 5.7 extend によって font-weight が異なる 2 つのコンポーネントを表示

styled 内で処理を実装したい場合

最後に少し複雑な処理を紹介します。

こちらは CSS の「color」に対して color プロパティを定義してから、無名関数を割り当てて Props を使った処理を実装しています。

app.js を次のように修正します。

▼リスト 5.6 ./src/app.js

```
import React, { Component } from 'react';
import styled from 'styled-components';
import logo from './logo.svg';
import './App.css';

const AppIntro = styled.p.attrs({
  fontSize: props => {
    if (props.order !== undefined) {
      const sizeList = ['small', 'large', 'xx-large'];
      return sizeList[props.order];
    }
    return '';
  }
})`
font-size: ${props => props.fontSize};
`;

class App extends Component {
  render() {
```

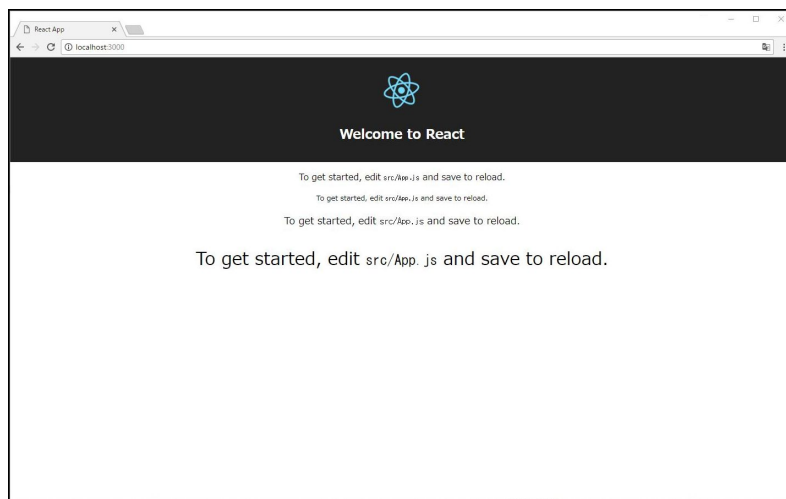
```

return (
  <div className="App">
    <header className="App-header">
      <img src={logo} className="App-logo" alt="logo" />
      <h1 className="App-title">Welcome to React</h1>
    </header>
    <AppIntro>
      To get started, edit <code>src/App.js</code> and save to reload.
    </AppIntro>
    <AppIntro order={0}>
      To get started, edit <code>src/App.js</code> and save to reload.
    </AppIntro>
    <AppIntro order={1}>
      To get started, edit <code>src/App.js</code> and save to reload.
    </AppIntro>
    <AppIntro order={2}>
      To get started, edit <code>src/App.js</code> and save to reload.
    </AppIntro>
  </div>
);
}
}

export default App;

```

修正後にブラウザを表示するとサイズ違いの文字が4つ表示されます。Propsによってサイズを設定できるようになりました。



▲図 5.8 attrs によって styled 内で処理を実装して表示

■コラム：styled-components のサーバサイドレンダリング (SSR)

styled-components ですが、v2.0 よりサーバサイドレンダリングが可能になりました。公式サイトアナウンス (<https://www.styled-components.com/releases#v2.0.0>)

著者が技術書典 4 で出したもう一冊「Get Ready for Next.js」にて「Next.js」と合わせて使う方法を解説しておりますので、ご興味がありましたら合わせて見てください。

あとがき

ここまで読んでいただき本当にありがとうございました。技術書典には一般参加していたのですが、「React 本があまりに少ない」という思いから初サークル参加して React の本を書きました。

書いてみると、「React ってあんまり書くこと無くね」という厳しい現実と戦う日々でした。仕様としてシンプルなのでテクニック集とすると、マニアックなことばかりになりやすく何度も考えて、次のことを書いて行こうとしました。

- 自分がやってきたことを書く
- 今まで使ってきたモジュールでいいものを紹介する
- できれば素振りがしやすい書き方にする
- React で楽しいと思ったことを書く

それを踏まえて各章を書いております。ただ、書ききれないこともありました。

- React v16.3.0 の新機能（特に ref 周りの新 API）
- styled-components を使った Grid Generator
- ゼロコンフィグ・モジュールバンドラー「poi」
- ゼロコンフィグ・モジュールバンドラーの機能比較
- react-helmet
- react-leaflet
- react-konva
- react-i18next

このへんについては、次回以降で取り上げたいと考えています。この本を読んでいた方が明日からお仕事で React を書く際に「れいなさんの本に出ていた書き方できれいに書けた」と進研〇ミの方式でコードが書けるようになったら幸いです。

次の技術書典もがんばるぞい。

お問い合わせ先

本書に関するお問い合わせ： megablacklabel@gmail.com

免責事項

本書に記載された内容は、情報の提供のみを目的としております。したがって、本書を用いた開発、制作、運用は、必ずご自身の責任と判断によって行ってください。

これらの情報による開発、制作、運用の結果について、著者はいかなる責任も負いません。

著者紹介

本文 れいな@底なし沼の魔女 / @MegaBlackLabel

底なし沼のほとりに住む魔女。React が大好きでついに同人誌を出すようになった。

表紙イラスト 紫楽 / @violet63davi

こだわり抜いた 1 品なので皆様のお気に召したら幸いです。

Libraries of React

2018 年 4 月 22 日 ver 1.0

著 者 れいな@底なし沼の魔女

イラスト 紫楽

印刷所 ねこのしっぽ

(C) 2018 底なし沼の魔女